



DESPLEGAR



SESIÓN 06 · UNIDAD II · ETHEREUM Y CONTRATOS

SOLIDITY I EL PRIMER CONTRATO

Hoy escriben un programa que, una vez desplegado, nadie puede cambiar. Ni siquiera ustedes.

Blockchain y Web 3.0 · Ingeniería de Sistemas
Universidad de San Buenaventura Medellín

CÓMO VAMOS A USAR LAS TRES HORAS

La sesión más práctica hasta ahora: al salir, cada pareja tiene un contrato propio desplegado y verificado en una red pública.

A

EL LENGUAJE

Estructura, tipos, funciones, visibilidad, errores, eventos, modificadores y el costo del storage.

~70 min

B

LABORATORIO 06 · REMIX

Registro de certificados: completar, probar, desplegar y verificar en Sepolia.

~80 min

C

ENTREGA DEL ANTEPROYECTO

Qué se entrega hoy, la prueba de la base de datos y cómo se filtra el alcance.

~30 min



AL SALIR DE AQUÍ

Escribir, desplegar y verificar un primer contrato funcional en una red de prueba.

LAS PREGUNTAS QUE LA SESIÓN RESPONDE

- 01** ¿Qué partes tiene un archivo de Solidity y para qué sirve cada una?
- 02** ¿Quién puede llamar cada función, y cuáles cuestan gas?
- 03** ¿Cómo falla bien un contrato, y cómo le cuenta al mundo lo que hizo?
- 04** ¿Qué datos nunca deben ir a una cadena pública?

RA3 · Programar contratos inteligentes en Solidity aplicando patrones y estándares de la industria.



SIGLAS Y TÉRMINOS DE HOY

SPDX*Software Package Data Exchange*

Identificador estándar de licencia en la primera línea. El compilador advierte si falta.

REMIX*Remix IDE*

Entorno de desarrollo en el navegador: editar, compilar, desplegar y llamar sin instalar nada.

MSG.SENDER*remitente del mensaje*

La dirección que llamó directamente a la función. La base de todo control de acceso.

REVERT*reversión*

Deshace todos los cambios de la llamada y devuelve un error. El gas consumido se paga igual.

PRAGMA*directiva del compilador*

Qué versiones de Solidity pueden compilar el archivo.

BYTECODE*código de bytes*

Lo que produce el compilador y se despliega: las instrucciones de la EVM de la Sesión 5.

VIEW · PURE*mutabilidad*

view lee el estado sin cambiarlo; pure ni lo lee. Llamadas desde fuera: sin costo.

EVENT*evento*

Registro que el contrato deja en el recibo. Barato, legible desde fuera, invisible para otros contratos.

A



APROXIMADAMENTE 70 MINUTOS

EL LENGUAJE

Solidity se parece a JavaScript en la sintaxis y a nada conocido en las consecuencias: cada línea se paga y ninguna se puede corregir después.



NI INTELIGENTE, NI CONTRATO

El término lo acuñó Nick Szabo en los años noventa. Hoy describe algo más modesto y más preciso:

Un contrato inteligente es un programa desplegado en la cadena, con dirección propia, que se ejecuta exactamente como fue escrito cuando alguien lo llama.

NO ES INTELIGENTE

No interpreta intenciones ni corrige errores. Si el código dice algo que el autor no quería, se ejecuta lo que dice el código. Varios de los robos más grandes de la historia fueron código funcionando exactamente como estaba escrito.

NO ES UN CONTRATO JURÍDICO

Puede automatizar el cumplimiento de un acuerdo, pero no es por sí mismo un contrato ante la ley. Su validez jurídica en Colombia se discute en la Sesión 16.



ANATOMÍA DE UN CONTRATO

```
CONTADOR.SOL

// SPDX-License-Identifier: MIT
pragma solidity 0.8.34;

contract Contador {
    uint256 public cuenta;           // estado

    event Incrementado(uint256 nueva); // evento

    function incrementar() external { // función
        cuenta += 1;
        emit Incrementado(cuenta);
    }
}
```

LICENCIA

Primera línea. Los contratos verificados son públicos: la licencia dice qué se puede hacer con ese código.

PRAGMA

Qué versión del compilador puede compilar el archivo. Protege de compilar con una versión con otras reglas.

CONTRACT

Como una clase: agrupa estado y funciones. Se despliega en su propia dirección.

ESTADO

Variables que viven en storage. Persisten y cuestan (Sesión 5).

EVENTOS Y FUNCIONES

Lo que el contrato expone y lo que cuenta al exterior.



PRAGMA: FIJAR CON QUIÉN SE COMPILA

ESCRITURA	QUÉ ACEPTA	CUÁNDO USARLA
<code>pragma solidity 0.8.34;</code>	Solo 0.8.34.	Contratos que se despliegan: el bytecode verificado debe ser reproducible. Es la que usamos en clase.
<code>pragma solidity ^0.8.34;</code>	0.8.34 o superior, dentro de 0.8.x.	Bibliotecas y material que otros reutilizan con otra versión.
<code>pragma solidity >=0.8.0 <0.9.0;</code>	Todo el rango indicado.	Código que otros compilarán con versiones distintas.



POR QUÉ IMPORTA LA 0.8

Desde la versión 0.8.0, una suma que se desborda revierte automáticamente. Antes daba la vuelta en silencio: $255 + 1$ en un uint8 daba 0. Ese cambio eliminó una familia entera de robos, como veremos en la Sesión 9.

Error típico al verificar en el explorador: elegir otra versión del compilador. El bytecode no coincide y la verificación falla.



LOS TIPOS QUE VAN A USAR TODO EL SEMESTRE

TIPO	QUÉ GUARDA	DETALLE QUE IMPORTA
uint256	Entero sin signo de 256 bits. Existen de uint8 a uint256.	El natural de la EVM. Usar tamaños menores solo ahorra al empaquetar en storage (Sesión 7).
int256	Entero con signo.	Poco usado: saldos y montos nunca son negativos.
bool	true o false.	Ocupa un byte en storage, no un bit.
address	Dirección de 20 bytes.	address payable, además, puede recibir ETH con transfer y send.
bytes32	Exactamente 32 bytes.	El tipo natural de un hash. Barato y de tamaño fijo.
bytes · string	Secuencias de tamaño variable.	Caros. string no tiene longitud ni comparación nativa: se convierte a bytes.

No existe float ni double. Los montos se guardan en la unidad más pequeña, como el wei (Sesión 5).



VALORES POR DEFECTO, LÍMITES Y COMPARACIONES

```
uint256 x;           // vale 0: toda variable nace en cero
bool b;              // vale false
address a;           // vale address(0)
uint8 y = 255;
y = y + 1;           // REVIERTE: desbordamiento (desde 0.8.0)

string memory p = "Sistemas";
// if (p == "Sistemas")           NO COMPILA
if (keccak256(bytes(p)) == keccak256("Sistemas")) { } // así sí
if (bytes(p).length == 0) { } // ¿vacío?
```

LA CONSECUENCIA DE LOS CEROS

No hay «null». Un certificado que nunca se emitió no da error: devuelve ceros. Por eso el laboratorio guarda un campo existe.

LA DE LOS STRINGS

Comparar textos exige hashearlos. Por eso en cadena se prefieren bytes32 e identificadores numéricos.



VARIABLES DE ESTADO Y VARIABLES LOCALES

```
contract Tienda {  
    uint256 public ventas;           // ESTADO · storage · persiste  
  
    function vender(uint256 precio) external {  
        uint256 conIva = precio * 119 / 100; // LOCAL · desaparece  
        ventas += conIva;                  // escritura en storage  
    }  
}
```

ESTADO

- Declarada fuera de las funciones.
- Vive en storage: la cadena la recuerda para siempre.
- Estrenar una ranura: 22 100 de gas (Sesión 5).

LOCAL

- Declarada dentro de una función.
- Vive en la pila o en memory: se borra al terminar.
- Casi gratis.



CUÁNTO CUESTA EL CONTRATO DE HOY

OPERACIÓN EN CERTIFICADOSUB	GAS MEDIDO	POR QUÉ
Desplegar	557 963	Guardar 2 221 bytes de código en la cadena, más el constructor.
Emitir el primer certificado	94 806	Estrena 2 ranuras del certificado y la del contador emitidos (0 → 1).
Emitir los siguientes	77 706	El contador ya existe: 1 → 2 cuesta 5 000 y no 22 100. Diferencia: 17 100.
Revocar	32 460	Cambia un bool en una ranura que ya existe, y emite un evento.
Cambiar el emisor	28 636	Sobrescribe una dirección.
Verificar (view)	0 para quien consulta	Es una lectura al nodo, no una transacción.

Con una tarifa base cercana a 1 gwei, como la de Sepolia en septiembre de 2026, desplegar cuesta del orden de 0,0006 ETH de prueba y emitir, 0,0001.

Error típico: «guardemos el PDF del diploma en el contrato». Un PDF de 100 KB son más de 3 000 ranuras: decenas de millones de gas.



VISIBILIDAD: CUATRO PALABRAS, CUATRO FRONTERAS

VISIBILIDAD	DESDE FUERA	DESDE ESTE CONTRATO	DESDE UN HIJO
<code>public</code>	Sí	Sí	Sí
<code>external</code>	Sí	Solo con <code>this.funcion()</code>	No directamente
<code>internal</code>	No	Sí	Sí
<code>private</code>	No	Sí	No



PRIVATE NO SIGNIFICA SECRETO

`private` solo impide que OTROS CONTRATOS la lean. Cualquier persona puede leer el storage completo de cualquier contrato con una llamada al nodo. Guardar una contraseña en una variable `private` es publicarla.

Una variable de estado `public` genera automáticamente una función de lectura con su nombre. Por eso se puede consultar `emisor()` en el laboratorio sin haberla escrito.



VIEW, PURE Y PAYABLE

MODIFICADOR	LEE ESTADO	CAMBIA ESTADO	RECIBE ETH	COSTO SI SE LLAMA DESDE FUERA
(ninguno)	Sí	Sí	No	Transacción: cuesta gas.
view	Sí	No	No	Llamada gratuita al nodo.
pure	No	No	No	Llamada gratuita al nodo.
payable	Sí	Sí	Sí	Transacción: cuesta gas.

«Gratuita» tiene letra pequeña:

- Es gratis cuando una aplicación la consulta directamente a un nodo: no se crea transacción.
- Si otro contrato la llama dentro de una transacción, el gas se paga igual: verificar() costaría unos 27 000.
- Una función sin payable que recibe ETH revierte: es una protección, no un descuido.



VARIABLES GLOBALES

VARIABLE	QUÉ CONTIENE	CUIDADO
<code>msg.sender</code>	Quién llamó directamente a esta función.	Si un contrato llama a otro, es el contrato intermediario, no la persona.
<code>msg.value</code>	Cuántos wei vinieron con la llamada.	Solo tiene sentido en funciones payable.
<code>block.timestamp</code>	Marca de tiempo del bloque, en segundos.	La elige el proponente del bloque dentro de un margen pequeño: no sirve para decisiones al segundo.
<code>block.number</code>	Altura del bloque.	No es una medida exacta de tiempo.
<code>tx.origin</code>	La cuenta externa que firmó la transacción original.	NUNCA para control de acceso. Es una vulnerabilidad clásica (Sesión 9).

Lo que NO hay: la hora real, archivos, internet, números aleatorios. Todo lo que el contrato sabe está en la cadena o viene en la llamada.

REQUIRE, REVERT, ERRORES PERSONALIZADOS Y ASSERT

```
// 1 · require con mensaje: simple, pero guarda el texto en el bytecode
require(msg.sender == emisor, "no autorizado");

// 2 · error personalizado: más barato y lleva datos
error NoEsEmisor(address quien);
if (msg.sender != emisor) revert NoEsEmisor(msg.sender);

// 3 · assert: para lo que NUNCA debería pasar
assert(totalEmitidos >= totalRevocados);
```

	PARA QUÉ	GAS
require / revert	Validar entradas, permisos y condiciones de uso.	Consumido hasta el error; el resto se devuelve.
error personalizado	Lo mismo, con datos estructurados que la interfaz puede mostrar.	Menor: no almacena cadenas de texto.
assert	Invariantes internas. Si falla, hay un error de programación.	Produce un Panic: señal de defecto, no de uso.



QUÉ VE QUIEN LLAMA CUANDO ALGO FALLA

01

LA BILLETERA SIMULA

Antes de firmar, estima el gas ejecutando la llamada. Si revierte, avisa: en Remix aparece «Gas estimation failed».

02

EL ERROR VIAJA CODIFICADO

NoEsEmisor(0xf39F...2266) viaja como 0x63898998 + la dirección: el selector del error y su argumento, igual que una función.

03

SI SE FIRMA IGUAL

La transacción entra a un bloque con Status Fail, se paga el gas consumido y no cambia nada. Es la evidencia del intento fallido del laboratorio.



ERROR TÍPICO

«Si revierte, no pasó nada». Para el estado es cierto. Para quien firmó, no: el gas consumido se cobra y el intento queda para siempre en la cadena.



EVENTOS

```
event CertificadoEmitido(bytes32 indexed hashDocumento, string programa, uint64 fecha);

emit CertificadoEmitido(hashDocumento, programa, ahora);
```

POR QUÉ EXISTEN

- Son la forma barata de dejar constancia: menos de la mitad que escribir en storage (medido en la Sesión 5).
- Las aplicaciones se suscriben a ellos para actualizarse sin consultar todo el tiempo.
- Quedan en el recibo de la transacción, para siempre.

LO QUE NO PUEDEN HACER

- Otro contrato NO puede leer un evento. Si la lógica lo necesita, va en storage.
- Hasta 3 parámetros indexed: son los que se pueden filtrar rápido.
- Un evento no prueba nada por sí mismo si el contrato que lo emite no es el esperado.

Regla práctica: toda función que cambia el estado emite un evento. Sin evento, la interfaz de la Sesión 12 no se entera.



CÓMO SE VE CERTIFICADOEMITIDO EN EL EXPLORADOR

PARTE DEL LOG	CONTENIDO	DE DÓNDE SALE
Address	La dirección de SU contrato.	Quién emitió: lo primero que se comprueba.
Topics[0]	0x774ca613...3fd6	keccak256("CertificadoEmitido(bytes32,string,uint64)")
Topics[1]	El hash del diploma, completo.	El parámetro indexed: por eso se puede buscar.
Data	«Ingenieria de Sistemas» y la fecha, codificados en ABI.	Los no indexados: se leen, no se filtran.

[] POR QUÉ EL HASH VA INDEXADO

Una empresa que recibe un diploma calcula su hash y pregunta al nodo: «dame los eventos CertificadoEmitido cuyo Topics[1] sea este hash». Sin indexar, tendría que descargar y leer todos los eventos del contrato. Indexar el programa no aporta: nadie busca por programa.



CONSTRUCTOR Y MODIFICADORES

```
address public emisor;

constructor() {
    emisor = msg.sender;    // UNA vez, al desplegar
}

modifier soloEmisor() {
    if (msg.sender != emisor) revert NoEsEmisor(msg.sender);
    _;                      // aquí se inserta la función
}

function emitir(bytes32 h, string calldata p) external soloEmisor {
    // solo llega aquí si pasó el modificador
}
```

- El constructor no queda en el bytecode desplegado: corre una vez y desaparece.
- Quien despliega es msg.sender en el constructor. Por eso el emisor es la cuenta con la que se despliega.
- El guion bajo _; marca dónde se ejecuta la función protegida.
- Un modificador olvidado en UNA función abre la puerta entera. Revisar cada función que cambia estado.



EL MAPPING QUE NECESITAMOS HOY

El laboratorio necesita buscar un certificado por su hash. Para eso existe el mapping: una tabla de pares clave → valor. Hoy basta con usarlo; la Sesión 7 lo estudia a fondo.

```
mapping(bytes32 => Certificado) private _certificados;  
  
_certificados[hash] = Certificado({ programa: p, fechaEmision: t, revocado: false, existe: true });  
  
Certificado storage c = _certificados[hash]; // referencia: los cambios se guardan
```

LO ÚNICO QUE HAY QUE SABER HOY

Toda clave existe: si nunca se escribió, devuelve el valor por defecto (ceros, falso, texto vacío). Por eso el certificado lleva el campo «existe»: sin él, no se distingue un certificado que nunca se emitió de uno emitido con datos vacíos.



LO QUE NUNCA VA A UNA CADENA PÚBLICA

La tentación es guardar el nombre del graduado, su cédula y su programa. Parece útil. Es un error grave, y no técnico: jurídico.

EL CHOQUE

La Ley Estatutaria 1581 de 2012 de protección de datos personales da a toda persona derecho a conocer, actualizar, rectificar y pedir la supresión de sus datos. Una cadena pública es inmutable: lo escrito no se puede suprimir.

LA SALIDA

En la cadena va solo el HASH del archivo del diploma. Quien tiene el PDF calcula el hash y verifica. Quien no lo tiene no aprende nada: un hash no identifica a una persona por sí solo.



REGLA PARA EL ANTEPROYECTO

Si su proyecto maneja datos de personas, el anteproyecto tiene que decir explícitamente qué va en la cadena, qué va fuera y por qué. Un proyecto que guarde datos personales en cadena no se aprueba.

EMITIR: VALIDAR, GUARDAR, AVISAR

```
function emitir(bytes32 hashDocumento, string calldata programa)
    external soloEmisor
{
    if (hashDocumento == bytes32(0)) revert HashVacio();
    if (bytes(programa).length == 0) revert ProgramaVacio();
    if (_certificados[hashDocumento].existe) revert YaEmitido(hashDocumento);

    uint64 ahora = uint64(block.timestamp);
    _certificados[hashDocumento] = Certificado(programa, ahora, false, true);
    emitidos += 1;

    emit CertificadoEmitido(hashDocumento, programa, ahora);
}
```

VALIDAR PRIMERO

Todas las comprobaciones antes de tocar el estado. Si algo falla, no se escribió nada.

CALLDATA

El texto se lee sin copiarlo a memoria: más barato (Sesión 5).

UN SOLO HASH, UNA VEZ

YaEmitido impide sobrescribir un certificado existente, incluido uno revocado.

EVENTO AL FINAL

Se emite cuando todo salió bien.

REVOCAR SIN BORRAR, VERIFICAR SIN PAGAR

```
function revocar(bytes32 h, string calldata motivo) external soloEmisor {
    Certificado storage c = _certificados[h];
    if (!c.existe) revert NoExiste(h);
    if (c.revocado) revert YaRevocado(h);
    c.revocado = true;
    emit CertificadoRevocado(h, motivo);
}

function verificar(bytes32 h) external view
    returns (bool valido, string memory programa, uint64 fecha, bool revocado)
{
    Certificado storage c = _certificados[h];
    return (c.existe && !c.revocado, c.programa, c.fechaEmision, c.revocado);
}
```

STORAGE, NO MEMORY

c apunta al dato guardado. Con memory se modificaría una copia y el cambio se perdería.

REVOCAR NO ES BORRAR

Queda la historia: existió, se emitió en tal fecha, se revocó por tal motivo. Eso es lo que da confianza.

VIEW

Quien verifica un diploma no paga nada ni necesita billetera.



PREGUNTA DE VERIFICACIÓN

Un compañero propone: «guardemos el nombre del graduado en una variable `private`; así nadie lo ve y cumplimos la Ley 1581». ¿Qué le responden?

- A** Bien: `private` oculta el dato a todos menos al contrato.
- B** Mal: `private` solo impide que otros contratos lo lean; cualquiera lee el storage, y además no se podría suprimir.
- C** Bien, siempre que el nombre se guarde en mayúsculas.

Un minuto en parejas. La respuesta correcta tiene dos razones: encuéntruelas las dos.

B



APROXIMADAMENTE 80 MINUTOS · EN PAREJAS

LABORATORIO 06 - REMIX

Completar el registro de certificados, probarlo en la máquina virtual del navegador, desplegarlo en Sepolia y verificarlo.



PREPARAR Y COMPILAR EN REMIX

01

ABRIR

app.remix.live, en el navegador donde está la billetera del curso.

02

CREAR ARCHIVO

File Explorer → carpeta contracts → nuevo archivo CertificadosUSB.sol. Pegar el andamiaje: andamiaje/s06/CertificadosUSB.sol.

03

COMPILADOR

Pestaña Solidity Compiler → desplegable Compiler: 0.8.34+commit.80d5c536. En Advanced Configurations, activar optimización con 200 ejecuciones.

04

COMPILAR

Compile CertificadosUSB.sol. Salida esperada: marca verde en el ícono del compilador y ningún error en rojo.



POR QUÉ COMPILAR ANTES DE ESCRIBIR

El andamiaje compila tal como viene. Si no compila antes de tocarlo, el problema es de configuración, no de su código.

COMPLETAR LOS SIETE TODO

TODO	QUÉ HACER	CÓMO PROBARLO EN LA REMIX VM
1 · <code>modificador</code>	Revertir si quien llama no es el emisor.	Cambiar de cuenta en Remix e intentar emitir: debe fallar con <code>NoEsEmisor</code> .
2 · <code>constructor</code>	Guardar al emisor y emitir <code>EmisorCambiado</code> .	Botón emisor: debe ser la cuenta que desplegó.
3 y 4 · <code>emitir</code>	Validar, guardar, contar y emitir el evento.	Emitir un hash de prueba y consultar emitidos: 1.
5 · <code>revocar</code>	Validar existencia y estado, revocar.	Revocar dos veces: la segunda falla con <code>YaRevocado</code> .
6 · <code>verificar</code>	Devolver válido = existe y no revocado.	Verificar antes y después de revocar.
7 · <code>cambiar emisor</code>	Rechazar la dirección cero y traspasar.	Traspasar y comprobar que la anterior ya no puede emitir.

Después de cada TODO: compilar, desplegar de nuevo en la Remix VM y probar a mano. Avanzar de uno en uno ahorra la mitad del tiempo de depuración.



LA REMIX VM, PASO A PASO

- | | | |
|-----------|------------------|--|
| 01 | ENTORNO | Deploy & Run → Environment: Remix VM. Aparecen cuentas de prueba con 100 ETH ficticios cada una. |
| 02 | DESPLEGAR | Deploy. En Deployed Contracts aparece CertificadosUSB con un botón por función. |
| 03 | LEER | Botones azules (view): emisor, emitidos, verificar. Responden al instante y no cuestan nada. |
| 04 | ESCRIBIR | Botones naranja (transacción): emitir, revocar, cambiarEmisor. Cada uno deja una línea en la terminal. |
| 05 | REVISAR | En la terminal, desplegar la transacción: status, gas usado y logs con el evento. |

Cómo saber que funcionó: verificar(hash) devuelve true, el programa y una fecha distinta de cero justo después de emitir.



CALCULAR EL HASH DEL DIPLOMA

El repositorio trae un diploma ficticio, sin datos de ninguna persona. Se calcula su hash, se altera un solo bit y se vuelve a calcular:

```
EN LA CARPETA LABORATORIOS-EVM
```

```
node scripts/s06/hash-documento.js scripts/s06/diploma-de-prueba.pdf  
keccak : 0x87daf6ef2771bae4d21b7fb938774a35b9bd4a1a6989dd253c6694e9dcbed455
```

```
node scripts/s06/alterar-un-byte.js scripts/s06/diploma-de-prueba.pdf  
node scripts/s06/hash-documento.js scripts/s06/diploma-de-prueba-alterado.pdf  
keccak : 0xc89d02a9a7f8b6660f3567945455456ed1e2b96cdd0637371b863b2c0f913951
```

01

EMITIR

Pegar el primer valor como hashDocumento en emitir().

02

VERIFICAR

verificar() con el segundo valor: no es válido. Un bit distinto, otro hash (Sesión 2).



DESPLEGAR EN SEPOLIA

- | | | |
|-----------|------------------|--|
| 01 | ENTORNO | Environment: la opción de billetera del navegador (Browser Extension / Injected Provider – MetaMask). Aceptar la conexión. |
| 02 | RED | Confirmar que Remix muestra Sepolia, identificador 11155111. Si dice otra red, cambiarla en la billetera. |
| 03 | DESPLEGAR | Deploy. En la billetera, antes de confirmar: red Sepolia, valor 0, costo del orden de 0,001 ETH de prueba. Confirmar. |
| 04 | ESPERAR | Unos 12 a 30 segundos. Copiar la dirección del contrato desde Deployed Contracts. |
| 05 | USAR | Emitir el hash del diploma, verificar ambos hashes, intentar emitir desde otra cuenta, revocar. |



ANTES DE CONFIRMAR CUALQUIER FIRMA

Leer qué red, a qué dirección y cuánto. No se firma nada cuyo contenido no se entienda.

VER EL EVENTO Y VERIFICAR EL CÓDIGO

- | | | |
|----|--------|---|
| 01 | BUSCAR | sepolia.etherscan.io → la dirección del contrato. Ver la transacción de creación y las de uso. |
| 02 | LOGS | Abrir la emisión → pestaña Logs: CertificadoEmitido, con el hash del diploma en Topics[1] (A.14). |
| 03 | VERIFY | Pestaña Contract → Verify and Publish: Solidity (Single file), v0.8.34+commit.80d5c536, licencia MIT, optimización Yes con 200. |
| 04 | PEGAR | El código exacto que se desplegó. Si todo coincide aparece la marca verde de verificado. |
| 05 | LEER | Read Contract → verificar(hash): cualquier persona, sin billetera, comprueba el diploma. |

La última prueba es la que importa: cualquiera, desde cualquier computador y sin cuenta, puede comprobar el diploma. Eso es lo que se construyó hoy.



LAS PRUEBAS AUTOMÁTICAS DEL ANDAMIAJE

El repositorio trae 18 pruebas que definen el comportamiento correcto. Sirven como plan B si la sala bloquea Remix, y como comprobación final para todos:

EN LA CARPETA LABORATORIOS-EVM

```
cp andamiaje/s06/CertificadosUSB.sol contracts/s06/CertificadosUSB.sol
npx hardhat test test/s06/CertificadosUSB.test.js
#   con el andamiaje sin tocar:  4 passing · 14 failing
#   con los siete TODO bien:    18 passing
```

CÓMO LEER LO QUE FALLA

- Cada prueba en rojo dice qué comportamiento falta.
- Resolver en orden: despliegue, emitir, revocar, cambiar emisor.
- Terminado = 18 passing.

CUIDADO

- El cp REEMPLAZA la solución de referencia en su copia local.
- Para recuperarla: `git checkout contracts/s06/CertificadosUSB.sol`.
- No suban la solución a su repositorio de entrega.



LO QUE SUELE SALIR MAL

SÍNTOMA	CAUSA
«Gas estimation failed» al emitir	Una validación revierte: hash en cero, programa vacío, hash ya emitido o cuenta distinta del emisor.
Remix no ve la billetera	Recargar Remix, aceptar la conexión en la billetera, mismo navegador. La billetera desbloqueada.
revocar no cambia nada	Usaron Certificado memory c en lugar de storage: modificaron una copia.
verificar dice válido un hash nunca emitido	Olvidaron comprobar existe.
Los certificados «desaparecieron»	Redesplegaron: cada Deploy crea un contrato nuevo y vacío.
La verificación en el explorador falla	Versión del compilador u optimización distinta a la del despliegue, o código distinto.

Si algo no se resuelve en 5 minutos: anotarlo, pedir ayuda y seguir con el siguiente paso.



EVIDENCIA DEL LABORATORIO 06

EVIDENCIA	DETALLE	PESO
Contrato verificado	Dirección en Sepolia con el código verificado en el explorador.	35 %
Tres transacciones	Hashes de: una emisión, una revocación y un intento fallido de emitir desde otra cuenta.	30 %
Prueba de alteración	Los dos hashes (original y alterado) y la captura de verificar() con cada uno.	20 %
Una respuesta	¿Por qué el contrato revoca en lugar de borrar? Un párrafo.	15 %



CÓMO SABER QUE TERMINARON

El contrato tiene la marca verde en el explorador; la pestaña Logs muestra CertificadoEmitido y CertificadoRevocado; hay una transacción con Status Fail desde otra cuenta; y Read Contract responde false para el hash alterado.



PREGUNTA DE VERIFICACIÓN

Revocaron un diploma por error. ¿Pueden volver a emitirlo con el mismo hash?

- A** Sí: revocar lo borra y el hash queda libre.
- B** No: el registro sigue existiendo y emitir revierte con YaEmitido. Hay que emitir un documento nuevo, con otro hash.
- C** Sí, si lo hace una cuenta distinta del emisor.

Pruébenlo en la Remix VM antes de responder: la respuesta está en una de las 18 pruebas.



C

 APROXIMADAMENTE 30 MINUTOS

ENTREGA DEL ANTEPROYECTO

Hoy se entrega la propuesta. La decisión más importante del semestre no es técnica: es qué queda dentro del alcance y qué queda fuera.



EL ANTEPROYECTO, EN TRES PÁGINAS

SECCIÓN	QUÉ DEBE RESPONDER
1 · Problema	Quién sufre qué, con qué frecuencia y cuánto le cuesta. Enunciado SIN nombrar la tecnología.
2 · Usuarios	Quiénes usan el sistema y qué hace cada uno.
3 · Por qué blockchain	El árbol de la Sesión 1, paso por paso, y la prueba de la base de datos (C.2).
4 · Qué va en cadena y qué no	En especial si hay datos personales (A.17 de hoy).
5 · Alcance	Historias que entran, con criterio de aceptación. Lista explícita de lo que queda fuera.
6 · Arquitectura preliminar	Contratos, interfaces y servicios externos, en un diagrama.
7 · Equipo	Tres integrantes y el rol inicial de cada uno.

Máximo tres páginas. Peso: 5 % de la nota final. Plantilla, rúbrica y ejemplos en [material/proyecto/01-anteproyecto.pdf](#).

¿UNA BASE DE DATOS LO RESOLVERÍA?

PASO DEL ÁRBOL (SESIÓN 1)	SI LA RESPUESTA ES...	ENTONCES
1 · ¿Más de una parte escribe?	No	Base de datos.
2 · ¿Intereses divergentes?	No	Base de datos compartida con control de acceso.
3 · ¿Hay un tercero aceptable para todos?	Sí	Usen ese tercero.
4 · ¿Datos personales?	Sí	En cadena solo el hash; el dato, fuera.
5 · ¿Tolera latencia y costo por operación?	No	Capa 2 o arquitectura convencional.
6 · ¿Es un problema de registro o de captura?	Captura	La cadena no garantiza que el dato sea cierto: ¿quién lo introduce?



LO QUE TIENE QUE DECIR EL ANTEPROYECTO

Qué alternativa sin cadena de bloques consideraron, y la razón concreta por la que no basta.

EL DOCENTE APRUEBA, RECORTA O DEVUELVE

DECISIÓN	CUÁNDO	QUÉ PASA DESPUÉS
Aprobado	Problema claro, justificación sólida, alcance que cabe en el semestre.	Se empieza a construir en la Sesión 7.
Aprobado con recorte	Buena idea, demasiado grande. Es lo más frecuente.	El docente marca qué historias salen. El equipo lo acepta o argumenta en 48 horas.
Devuelto	Se enuncia desde la tecnología, una base de datos lo resolvería mejor, o hay datos personales en cadena.	Nueva versión para la Sesión 7. No se pierde el proyecto, se pierde una semana.

LAS TRES PREGUNTAS QUE SE APLICAN PRIMERO

¿Se puede enunciar el problema sin nombrar la tecnología? ¿Qué alternativa sin cadena de bloques se consideró? ¿Cabe el alcance —lo que entra y lo que no— en una página? Si alguna respuesta es no, el anteproyecto vuelve.



LA RÚBRICA DEL ANTEPROYECTO

CRITERIO	PESO	LO QUE DISTINGUE UN 5,0
Problema y usuarios	20 %	Se enuncia sin la tecnología, con un caso concreto: quién, qué, cuánto le cuesta.
Por qué blockchain	30 %	Recorre los seis pasos y descarta la alternativa centralizada con un argumento propio.
Qué va en cadena	10 %	Separa dato y hash; nada personal en cadena; dice quién introduce cada dato.
Alcance	25 %	4 a 6 historias con criterio de aceptación, lista de fuera y el supuesto más riesgoso.
Arquitectura y equipo	15 %	Diagrama coherente con las historias; tres roles con responsable.

Devolución automática, sin importar el puntaje: datos personales en cadena, más de tres páginas, o ninguna lista de lo que queda fuera.



EL MISMO TEMA, BIEN Y MAL PLANTEADO

MAL PLANTEADO

- «Plataforma blockchain para historias clínicas de todas las EPS de Colombia.»
- Nombra la tecnología en el problema.
- Un solo escritor por historia: una base de datos basta.
- Datos de salud en cadena: choca con la Ley 1581.
- Sin lista de lo que queda fuera.

BIEN PLANTEADO

- «Tres semilleros de universidades distintas manejan un fondo común y ninguno acepta que otro lo administre solo.»
- Varios escritores con intereses divergentes, sin tercero aceptable.
- En cadena: montos y aprobaciones. Fuera: facturas y nombres.
- Fuera de alcance: pagos en pesos, app móvil.

Los dos ejemplos completos, desarrollados sección por sección, están en la plantilla.



LA PREGUNTA DE LA SEMANA

El contrato confía en una sola cuenta. Si roban esa clave, ¿qué puede hacer el ladrón, qué no, y cómo lo diseñarían para limitar el daño?

PARA LA PRÓXIMA SESIÓN

- Entregar la evidencia del laboratorio 06 antes de la Sesión 7.
- Completar las lecciones 1 y 2 de CryptoZombies.
- Leer en la documentación de Solidity las secciones sobre mapping y struct: son la base de la Sesión 7.
- Esperar la respuesta del docente sobre el anteproyecto y, si hay recorte, aceptarlo o argumentar en 48 horas.



LO QUE SE LLEVAN DE HOY

Hoy desplegaron un programa que ya no pueden cambiar. Por eso se valida todo antes de desplegar.

Solidity se aprende rápido. Lo difícil es acostumbrarse a que no hay parche posible: la próxima sesión empieza a construir con patrones que existen justamente por eso.

SESIÓN 7 · SOLIDITY II · ESTRUCTURAS DE DATOS Y PATRONES DE DISEÑO